EE426 / 626 Microprocessor Applications

General-Purpose I/O



EE426 / 626 Microprocessor Applications

STM32F446 General-Purpose Input / Output (GPIO)

Up to 16 I/O pins under control

Output state can be Push-pull or Open Drain with optional Pull-up or Pull-down

Data can come from output data register (GPIOx_ODR) or from a peripheral (as an alternate function)

Speed selection on a per I/O basis

Input state can be Floating, Pull-up or Pull-down, or Analog

Input data can go to input data register (GPIOx_IDR) or to a peripheral (as an alternate function)

Bitwise set and reset access to GPIOx_ODR

Analog function

Tast toggle capable at rate of once per two clock cycles

Highly flexible pin multiplexing capability

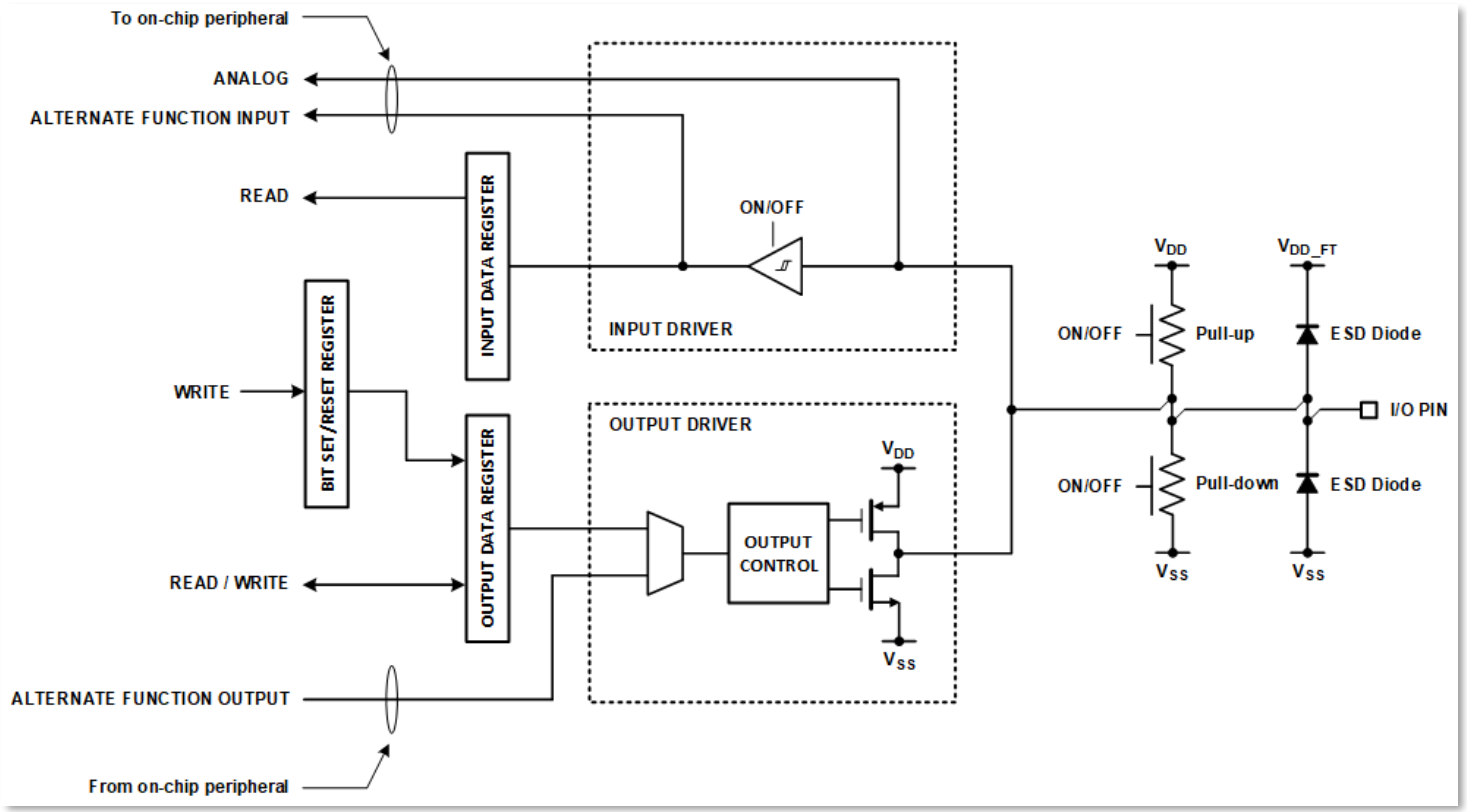
See RM0390 – STM32F446xx Advanced ARM Based 32-Bit MCUs

Chapter 7 General-purpose I/Os (GPIO)



EE426 / 626 Microprocessor Applications

STM32F446 General-Purpose Input / Output (GPIO)



EE426 / 626 Microprocessor Applications

STM32F446 General-Purpose Input / Output (GPIO)

Step 1. Turn on the clock to the GPIOA peripheral

```
RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;          // (1 << 0)
```

Step 2. Set pin 5's mode bits to "general-purpose output.“
Each pin gets 2 bits in MODER (00=input, 01=output, 10=alternate function, 11=analog)
So clear the existing 2 bits for pin 5, then write 01 into them.

```
GPIOA->MODER  &= ~GPIO_MODER_MODER5;          // ~(0x3 << (5 * 2))
GPIOA->MODER  |=  GPIO_MODER_MODER5_0;        // (0x1 << (5 * 2))
```

Step 3. Choose push-pull output type rather than open-drain, so the pin can actively drive both high and low
(needed to source current into the LED). This is the reset default, so the line just makes the choice explicit.

```
GPIOA->OTYPER  &= ~GPIO_OTYPER_OT_5;          // ~(0x1 << 5)
```



EE426 / 626 Microprocessor Applications

STM32F446 General-Purpose Input / Output (GPIO)

Step 4. Set the output speed. This controls slew rate; a simple LED has no high-frequency switching concern, so low speed is fine.

```
GPIOA->OSPEEDR &= ~GPIO_OSPEEDER_OSPEEDR5;  // ~(0x3 << (5 * 2))
```

Step 5. Disable any internal pull-up or pull-down resistor, since the output driver itself controls the logic level.

```
GPIOA->PUPDR   &= ~GPIO_PUPDR_PUPDR5;       // ~(0x3 << (5 * 2))
```

Step 6. Drive the pin high to turn the LED on.

Write to BSRR is preferred over ODR. It's an atomic set-only operation that doesn't disturb the other 15 pins on the port.

```
GPIOA->BSRR = GPIO_BSRR_BS5;                // (1 << 5)
```